Charters and Heuristics

0:09/6:55

Introduction

Imagine approaching an application with a mission rather than a script. Instead of simply following predefined steps, you observe how the software responds, ask "what if?" questions, and investigate unexpected behavior as you discover it. This is exploratory testing. Although it encourages curiosity, it is far from random. Charters provide a clear goal, while heuristics offer practical strategies that keep each testing session focused and productive.

This reading explains what exploratory testing is and introduces the core tools you will use to guide your sessions: charters and heuristics.

By the end of this reading, you will be able to:

- Define exploratory testing and explain how it differs from scripted testing
- Describe the purpose of a charter and identify its key components
- Apply common testing heuristics — including Goldilocks, Interruption, Rapid Input, Follow the Data, and SFDIPOT (Structure, Function, Data, Interfaces, Platform, Operations, Time) — to guide an exploratory session

Defining Exploratory Testing

Exploratory testing is a style of software testing that emphasizes the tester's personal freedom and responsibility. It is a creative and intuitive process where you conduct three activities at the same time: **learning** the system, **designing** test cases, and **executing** those tests. Unlike scripted testing, where test design and execution are separate, a script is written first and then followed. Here, your next test is influenced by what you discovered in the previous one.

Think of it as exploring a new city. You might have a general plan, like visiting a specific neighborhood, but you do not have a turn-by-turn script. As you walk around, you might see an interesting side street and decide to investigate. What you find there determines where you

go next. This is exactly how an exploratory tester approaches software—with a mission, but also with the freedom to follow their curiosity.

Staying Focused with Charters

Because exploratory testing is so open-ended, there is a risk of getting lost or testing features that are not a priority. This is why we use **charters** to provide focus. A charter is a clear, concise mission for your testing session. It is not a script, but a goal. A QA Lead or manager will typically provide you with a charter before you begin.

A good charter usually follows this format: **Explore [Target System] with [Resources] to discover [Information]**.

Here are a few examples:

- "Explore the **user registration page** with a **list of common invalid inputs** to discover **potential security vulnerabilities**."
- "Explore the **shopping cart checkout flow** with **different user accounts** to discover **issues related to state management**."

When you receive a charter, it sets the boundaries for your session. Your goal is to fulfill that mission within a set time, typically a short, focused block like 15 or 30 minutes. The charter ensures that your exploration is aligned with the team's immediate priorities.

Using Heuristics to Find Bugs

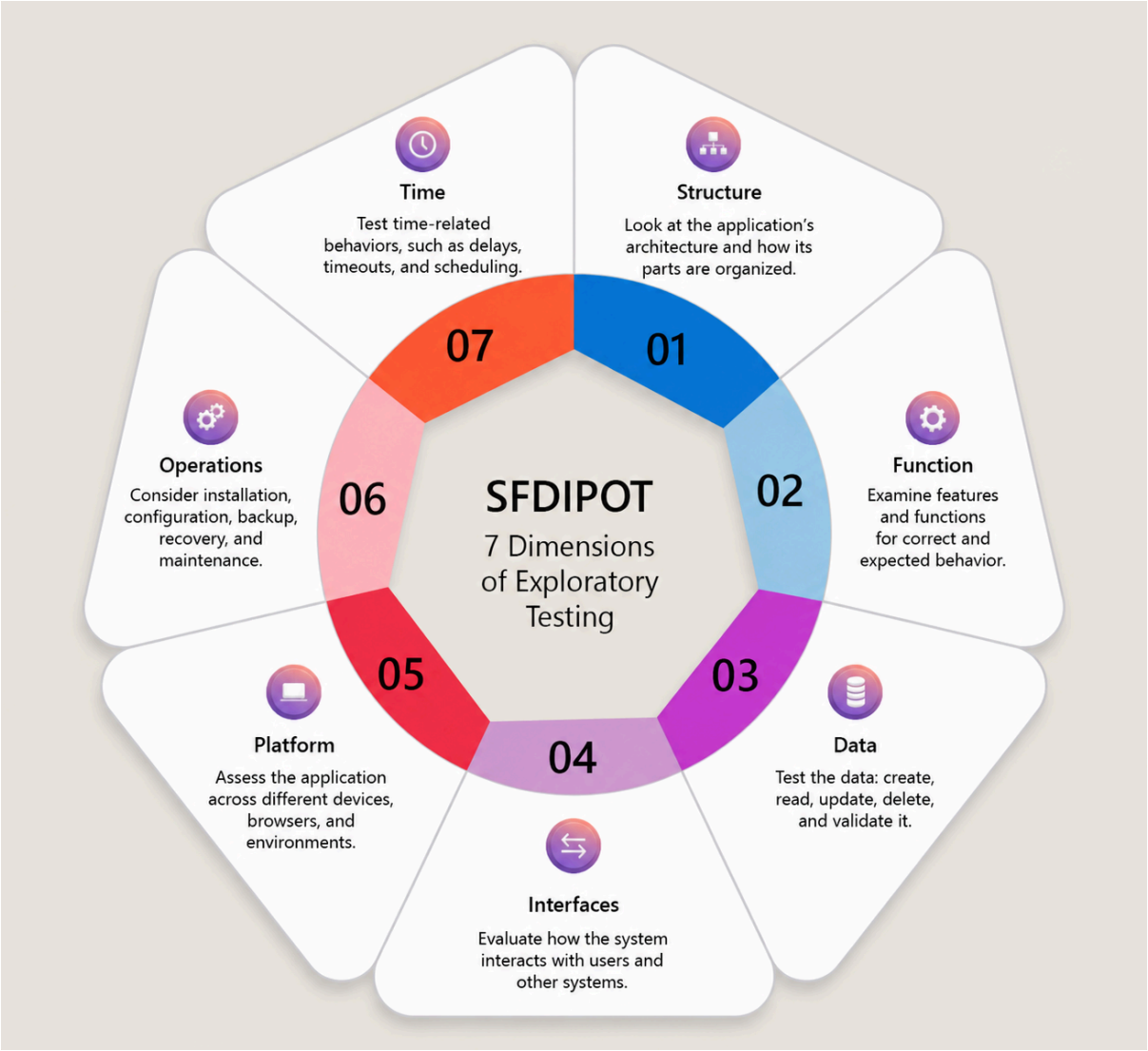
While a charter tells you where to look, **heuristics** are rules of thumb or simple strategies that help you decide how to look. They are mental shortcuts that guide you to common problem areas in software. Instead of trying to think of every possible test case from scratch, you can apply these proven patterns to uncover defects more efficiently.

There are many testing heuristics, but some of the most common and effective ones include:

- **Goldilocks Testing:** This focuses on testing values at, below, and above expected limits to uncover edge-case defects. For any input field, you would test the

boundaries. For example, if a field accepts numbers from 1 to 100, you should test 0, 1, 100, and 101. This helps find "off-by-one" errors.

- **Interruption Testing:** Modern users are constantly multitasking. What happens to the application if you are in the middle of a process and you get a phone call, switch to another app, or your network connection drops? Heuristics focused on interruption test the application's ability to handle these real-world scenarios gracefully.
- **Rapid Input:** Users are not always patient. Try clicking a "Submit" button multiple times in quick succession. Does it send multiple duplicate requests? Does the application crash? This simple heuristic often uncovers weaknesses in how an application handles user interactions.
- **Follow the Data:** Think about how data moves through the system. Where is it created, read, updated, and deleted (CRUD)? Try to perform these actions out of the expected order. For example, try to update an item before it has been created or delete it twice.
- **SFDIPOT**: SFDIPOT is often used in exploratory testing. It helps testers quickly understand and analyze product coverage. This heuristic test strategy assists in breaking down complex applications into smaller, testable perspectives to ensure comprehensive test coverage.



This image presents the SFDIPOT framework, which organizes the seven dimensions of exploratory testing into a circular diagram. At the center, the title reads "SFDIPOT – 7 Dimensions of Exploratory Testing." Surrounding the center is a seven-segment ring numbered 01 through 07, with each numbered segment corresponding to one testing dimension displayed in a panel around the outside of the circle. 01, Structure, encourages testers to look at the application's architecture and how its parts are organized. 02, Function, focuses on examining features and functions to ensure they behave correctly and as expected. 03, Data, emphasizes creating, reading, updating, deleting, and validating data. 04, Interfaces, instructs testers to evaluate how the system interacts with users and other systems. 05, Platform, highlights assessing the application across different devices, browsers, and environments. 06, Operations, covers installation, configuration, backup, recovery, and maintenance activities. 07, Time, focuses on testing time-related behaviors such as delays, timeouts, and scheduling. Each section includes a corresponding icon and is color-coded within the circular ring, visually reinforcing that effective exploratory testing considers all seven dimensions together to achieve comprehensive test coverage.

Conclusion

Exploratory testing combines the structure of a **charter** with the creative guidance of **heuristics**. A charter gives you a defined mission, ensuring your testing is focused and relevant. Heuristics give you a powerful toolkit of strategies to uncover hidden bugs that scripted tests would miss. By mastering these two concepts, you can transform your testing from a checklist activity into a dynamic and valuable investigation.